

Algorithm II: Predictive Propensity — Technical Strategy Document | FirstKnock Sales OS

ALGORITHM II: PREDICTIVE PROPENSITY

Technical Strategy Document — FirstKnock Sales OS | PropTech Field Sales Platform

Version 1.0 | March 2026

CLASSIFICATION: INTERNAL — ENGINEERING & PRODUCT TEAM ONLY

This document constitutes the full technical specification for Algorithm II: Predictive Propensity, the core intelligence layer of the FirstKnock Sales OS. It is intended for senior engineers, ML practitioners, and product leads responsible for implementation. All mathematical formulations, architecture decisions, and implementation priorities contained herein are derived from peer-reviewed research and validated PropTech industry practice.

EXECUTIVE SUMMARY

Algorithm II: Predictive Propensity is the intelligence engine that transforms FirstKnock from a digital door-knocking tool into a precision-guided field sales platform. Where unscored door-to-door canvassing yields conversion rates of 1–5%, Algorithm II targets the highest-propensity homeowners in the highest-density revenue zones, with routes engineered for maximum qualified conversations per shift. The algorithm ingests 140M+ property records from the RentCast API, processes them through a multi-stage machine learning pipeline, and delivers a ranked, routed, and dynamically calibrated daily work order to every field representative.

The Four Pillars

- **Predictive Feature Engineering** — Transforms raw RentCast property records into high-signal composite features: ownership duration decay, neighborhood absorption rates, equity band scoring, distress signal stacking, and life event proxies. An XGBoost gradient-boosted model outputs probabilistic sellability scores on a 0–1,000 normalized scale.
- **Dynamic Weight Calibration** — A Bayesian Beta-Binomial framework continuously updates propensity weights as field reps log funnel outcomes (knocked rightarrow answered rightarrow interested rightarrow converted). Thompson Sampling balances exploration of new weight configurations against exploitation of proven ones. Hidden Markov Models detect market regime shifts and trigger conditional weight adjustments.
- **Geospatial Clustering for Revenue Density** — HDBSCAN with Haversine metrics groups properties into density-optimized clusters. Propensity-weighted K-Means and REDCAP/MDD algorithms enforce contiguous, capacitated territories constrained to 40–60 houses per rep per day. Getis-Ord G_i^* hotspot analysis distinguishes statistically significant revenue concentrations from noise.

- TSP/Routing Optimization — 2-Opt edge-swap refinement generates loop-topology routes that guarantee tour lengths within 1.5times optimal (Christofides bound). VRPTW formulations inject virtual break nodes. Reinforcement learning (RL-AVNS) enables real-time route adaptation without full recalculation. Biomathematical Fatigue Constraints (BFCs) are integrated into the scheduling layer.

Expected Business Impact

25–40%Target Conversion Rate Lift vs. Unscored Knocking	15–20%Target Route Distance Reduction vs. Nearest-Neighbor	20–30%Target Qualified Conversations per Shift Increase	82.33%Validated Predictive Conversion Accuracy (Research Benchmark)
--	---	--	---

Implementation Roadmap Summary

Table 1 — Algorithm II Implementation Phases

Phase	Timeline	Deliverables	Primary Algorithms
Phase 1: Foundation	Weeks 1–6	RentCast feature pipeline, baseline propensity scoring, DBSCAN/K-Means clustering, 2-Opt routing	Weighted sum scoring, DBSCAN, K-Means, 2-Opt
Phase 2: Intelligence Layer	Weeks 7–14	XGBoost/LightGBM model, Bayesian calibration, propensity-weighted clustering, VRPTW break injection, real-time route API	XGBoost, Beta-Binomial MAB, HDBSCAN, VRPTW, OR-Tools
Phase 3: Continuous Learning	Weeks 15–24	Full MLOps pipeline, market regime detection, fatigue modeling, A/B testing framework	HMM, ADWIN/PSI, BFCs, RL-AVNS, Thompson Sampling

1. SYSTEM ARCHITECTURE OVERVIEW

The Algorithm II pipeline is a six-stage sequential architecture. Each stage produces a structured output consumed by the next, with feedback loops from the field rep application back into the Bayesian calibration layer. The pipeline is designed for daily batch execution with real-time delta updates triggered by field rep interactions.

1. RentCast API Ingestion — Pull 140M+ property records via RentCast REST API. Extract top-15 high-signal fields (ownership, LTV, DOM, absorption rate, property condition, life event flags). Store in PostGIS-enabled PostgreSQL with geospatial indexing on lat/lon coordinates.
2. Feature Engineering Layer — Apply time-based decay functions (exponential ownership duration), compute rolling neighborhood context windows (30/60/90-day absorption rates, DOM trends), construct composite motivation index (equity band + distress stack + life event proxy), and generate interaction features from RentCast field intersections.
3. Propensity Scoring Engine — Process engineered features through XGBoost gradient-boosted model. Output probabilistic sellability score (0–1,000 normalized scale, 500 = median). Apply Bayesian Beta-Binomial calibration weights updated from prior field rep feedback cycles.
4. Geospatial Clustering Module — Apply HDBSCAN with Haversine metric to group scored properties into density-based clusters. Execute Getis-Ord Gi* hotspot analysis via PySAL to identify statistically significant revenue concentrations. Apply REDCAP/MDD contiguous zone enforcement with 40–60 house/day capacity constraints.
5. Route Optimization Engine — Generate baseline routes via nearest-neighbor construction heuristic. Refine with 2-Opt edge swaps (O(1) symmetric evaluation). Enforce loop topology (Christofides guarantee < 1.5times optimal). Inject virtual break nodes via VRPTW formulation using Google OR-Tools.
6. Field Rep Application Delivery — Push daily work order (ranked property list + optimized route + propensity scores) to rep mobile app. Capture funnel progression events (knocked/answered/interested/converted) as real-time feedback into Bayesian calibration layer.

1.2 Technology Stack

Table 2 — Algorithm II Technology Stack with Version Targets

Layer	Component	Library / Tool	Version Target	Purpose
Data Ingestion	RentCast API Client	requests / httpx	geq 2.31	Property record extraction, 140M+ records
Data Storage	Geospatial Database	PostGIS / PostgreSQL	geq 15.0	Lat/lon indexing, spatial queries

Feature Engineering	Tabular Processing	Pandas / NumPy	geq 2.0 / 1.26	Decay functions, rolling windows, LTV calc
Feature Engineering	Spatial Features	GeoPandas / PySAL	geq 0.14 / 23.x	Absorption rates, Getis-Ord Gi* hotspots
Propensity Scoring	Gradient Boosting	XGBoost / LightGBM	geq 2.0 / 4.x	Probabilistic sellability scoring
Propensity Scoring	ML Utilities	Scikit-learn	geq 1.4	Preprocessing, validation, HDBSCAN, K-Means
Bayesian Calibration	Statistical Computing	SciPy / NumPy	geq 1.12 / 1.26	Beta-Binomial updates, Thompson Sampling
Market Regime Detection	HMM Modeling	hmmlearn	geq 0.3	Market state inference (hot/cold/neutral)
Geospatial Clustering	Density Clustering	hdbscan (scikit-learn)	geq 0.8.33	HDBSCAN with Haversine metric
Territory Design	Contiguous Zoning	REDCAP / PySAL	geq 23.x	Capacitated contiguous zone enforcement
Route Optimization	TSP / VRPTW Solver	Google OR-Tools	geq 9.8	2-Opt, VRPTW, virtual break nodes
MLOps	Drift Detection	Evidently / Metaflow	geq 0.4 / 2.x	PSI tracking, retraining triggers
MLOps	Experiment Tracking	MLflow	geq 2.10	Model versioning, A/B weight configs

1.3 Data Refresh Cadence and Latency Requirements

Table 3 — Data Refresh Cadence and Latency SLAs

Pipeline Stage	Refresh Cadence	Max Acceptable	Trigger Condition
RentCast API Pull	Daily (02:00 local)	< 4 hours full refresh	Scheduled cron + on-demand delta
Feature Engineering	Daily post-ingestion	< 90 minutes	Triggered by ingestion completion
Propensity Scoring	Daily post-features	< 30 minutes (Mini-batch)	Triggered by feature pipeline completion
Geospatial Clustering	Weekly full re-cluster	< 60 minutes	Weekly schedule + AUC drift trigger
Route Optimization	Daily pre-shift (05:30)	< 15 minutes per rep	Scheduled + real-time delta on conversion event
Bayesian Weight Update	Real-time (event-driven)	< 5 seconds per event	Field rep funnel event logged
Model Retraining	Performance-triggered	< 6 hours full retrain	AUC drop below 0.72 or PSI > 0.25

2. PREDICTIVE FEATURE ENGINEERING

Feature engineering is the highest-leverage component of Algorithm II. The quality of the propensity score is entirely determined by the signal richness of the input features. This section specifies every engineered feature, its mathematical formulation, implementation pattern, and expected contribution to model performance.

2.1 Ownership Duration Decay Functions

Ownership duration is a primary predictor of selling propensity. The probability of sale is not linear with time — it follows a hazard function that peaks at specific ownership milestones (5–7 years, 10–12 years) and decays between them. Two complementary mathematical frameworks are implemented.

Exponential Decay Baseline: The foundational decay feature applies $f(t) = e^{(-\lambda t)}$ where t is ownership duration in months and λ is the decay rate parameter. Recommended λ values by property segment are as follows: $\lambda = 0.008$ for standard residential (half-life approx 87 months), $\lambda = 0.012$ for investment/rental properties (higher turnover expectation, half-life approx 58 months), and $\lambda = 0.005$ for luxury/high-equity properties (lower turnover, half-life approx 139 months). This function outputs a continuous score in $[0,1]$ where values approaching 1.0 indicate recent acquisition (lower propensity) and values approaching 0.0 indicate very long ownership (higher propensity for life-event-driven sale).

Cox Proportional Hazards Adjustment: The Cox model extends the baseline decay by

incorporating property-specific covariates: $h(t|X) = h(t) \exp(\beta_{loc} \cdot location_score + \beta_{area} \cdot floor_area + \beta_{lvr} \cdot ltv_ratio + \beta_{life_event} \cdot life_event_flag)$. Research confirms that poor location scores and larger floor areas decrease the odds of a quick sale, while high LTV ratios and life event flags increase hazard. The Cox model output (partial hazard score) is used as a multiplicative adjustment to the exponential decay baseline, producing the final ownership_duration_propensity feature.

Implementation Note: The disposition effect and loss aversion must be encoded as a non-linear adjustment. Owners with unrealized losses (current AVM < purchase price) exhibit significantly reduced selling propensity regardless of ownership duration. Apply a loss_aversion_penalty = max(0, (purchase_price - current_avm) / purchase_price) as a subtractive modifier to the decay score.

2.2 Neighborhood Context Features

Neighborhood context features capture the macro-environment in which a property sits. These features are computed at the ZIP code and census tract level, then joined to individual property records. All rolling windows are computed over the prior 30, 60, and 90 days to capture both current state and momentum.

Table 4 — Neighborhood Context Features with Formulas and Rolling Windows

Feature Name	Formula / Derivation	Window	Market Signal
absorption_rate_30d	Units Sold / Total Active Inventory times 100	30-day	> 20% = Seller's Market; < 15% = Buyer's Market
absorption_rate_delta	absorption_rate_30d - absorption_rate_90d	30d vs 90d	Positive delta = accelerating demand
median_dom_zip	Median(days_on_market) for ZIP, trailing 60d	60-day	Lower DOM = higher urgency environment
dom_trend_slope	Linear regression slope of median DOM over 90d	90-day	Negative slope = market tightening
list_to_sale_ratio	Median(sale_price / list_price) for ZIP, trailing 60d	60-day	> 1.0 = bidding wars; < 0.95 = soft market
price_momentum_index	((median_sale_price_30d / median_sale_price_90d) - 1) times 100	30d vs 90d	Positive = rising prices, seller incentive
new_permit_velocity	Count(new_construction_permits) / ZIP_area_sqmi, trailing 90d	90-day	High permits = neighborhood investment signal

school_rating_zscore	Z-score normalization of school district rating within MSA	Static	Higher rating = stable demand, lower urgency
employment_rate_delta	Current employment rate " 12-month prior employment rate	YoY	Declining employment = distress signal
seasonal_adjustment_factor	MSA-specific seasonal index (400+ metro areas calibrated)	Monthly	Dampens spring signals in winter months

Normalization Approach: All neighborhood features are normalized using a RobustScaler (median and IQR-based) rather than StandardScaler, as real estate distributions are heavily right-skewed with outliers. Features are normalized within MSA boundaries, not nationally, to preserve geographic signal. The normalized features are prefixed with n_ in the feature store (e.g., n_absorption_rate_30d).

2.3 Composite Motivation Index

The Composite Motivation Index (CMI) is a single continuous score [0, 1] that aggregates equity position, financial distress signals, and life event proxies into a unified measure of seller urgency. Higher CMI values indicate owners who genuinely need to sell quickly and are more likely to accept below-market offers.

Equity Band Engineering (Continuous): The Loan-to-Value ratio is the primary equity feature. $LTV = \text{Outstanding Mortgage Balance} / \text{Current AVM}$. Research confirms that owners with higher LTV ratios set higher asking prices, experience longer DOM, and ultimately receive higher sale prices — indicating reduced urgency. The equity_band_score is computed as: $\text{equity_band_score} = 1 - \min(1, \max(0, (LTV - 0.20) / 0.80))$. This maps $LTV = 0.20$ (80% equity) to score 1.0 (high urgency potential) and $LTV = 1.00$ (zero equity) to score 0.0. Underwater properties ($LTV > 1.0$) receive a separate distress_flag = 1.

Distress Signal Stacking: The Wholesale Score proxy is constructed by stacking binary distress indicators with learned weights. The raw distress stack is: $\text{distress_raw} = w_{\bullet} \cdot \text{tax_delinquency_flag} + w_{\bullet} \cdot \text{foreclosure_notice_blat} + w_{\bullet} \cdot \text{court_violation_flag} + w_{\bullet} \cdot \text{vacancy_indicator} + w_{\bullet} \cdot \text{underwater_ltv_ratio} + w_{\bullet} \cdot \text{missed_payment_count}$. Recommended initial weights (to be calibrated via XGBoost feature importance): $w_{\bullet} = 0.25, w_{\bullet} = 0.30, w_{\bullet} = 0.10, w_{\bullet} = 0.15, w_{\bullet} = 0.15, w_{\bullet} = 0.15$. The distress_raw score is normalized to [0, 1,000] to produce the Wholesale Score analog, consistent with Leadflow's published scoring methodology.

Life Event Proxy Construction: Life events are encoded as time-decayed binary flags. Each flag is multiplied by a recency decay: $\text{life_event_score} = \sum (\text{event_flag}_i \times e^{-(\text{days_since_event}_i / 105)}$. Events included: divorce_filing (weight 1.0), job_relocation_indicator (weight 0.8), probate_filing (weight 0.9), new_dependent_flag (weight 0.4), retirement_age_proxy (owner

age > 65, weight 0.6), recent_marriage_record (weight 0.3). The decay constant lambda = 0.05 gives a half-life of approximately 14 days, ensuring recency is strongly rewarded.

Final Composite Motivation Index Formula: $CMI = \alpha \cdot equity_band_score + \beta \cdot normalize(distress_raw) + \gamma \cdot life_event_score + \delta \cdot ownership_duration_propensity$. Recommended initial weights: alpha = 0.30, beta = 0.25, gamma = 0.25, delta = 0.20. These weights sum to 1.0 and are subject to Bayesian calibration. The CMI is the single most important input feature to the XGBoost propensity model.

Key Insight: Advanced propensity scoring systems analyzing 800+ signals achieve scores normalized on a 0–1,000 scale (500 = median), where higher scores indicate greater propensity to sell within 90 days. The CMI is designed to be the primary driver of this score, with neighborhood context and ownership decay as secondary contributors.

2.4 RentCast API Feature Extraction

The following table specifies the top 15 highest-signal fields from RentCast property records, their derived feature names, interaction feature recommendations, and estimated feature importance ranking based on analogous PropTech ML deployments. Fields are extracted via the RentCast Properties and Market endpoints.

Table 5 — Top 15 RentCast API Fields: Feature Extraction and Interaction Recommendations

Rank	RentCast Field	Derived Feature Name	Transformation	Interaction Recommendation
1	lastSaleDate	ownership_duration_months	Months since last sale date	times ltv_ratio → equity_duration_int
2	estimatedValue / lastSalePrice	unrealized_gain_pct	(AVM - purchase) / purchase times 100	interaction times ownership_duration →
3	loanAmount / estimatedValue	current_ltv_ratio	Direct ratio calculation	times tax_delinquency →
4	propertyType	property_type_encoded	One-hot encoding (SFR/MFR/Condo/Land)	distress_equity_stack times absorption_rate →
5	daysOnMarket (if listed)	dom_current	Raw days, log-transformed	times list_to_sale_ratio
6	taxAssessedValue / estimatedValue	assessment_gap_ratio	Tax assessed / AVM	→ times urgency_signal ownership_duration → undervaluation_signal

7	squareFootage	size_zscore_msa	Z-score within MSA	times absorption_rate
8	yearBuilt	property_age_years	Current year "yearBuilt	rightarrow times size_demand_fit maintenance_cost _proxy rightarrow
9	bedrooms / bathrooms	bed_bath_ratio	Bedrooms / Bathrooms	times median_dom_zip
10	ownerOccupied	owner_occupied_flag	Binary 0/1	rightarrow times layout_market_fit life_event_score rightarrow
11	rentEstimate / estimatedValue	gross_rent_yield	Annual rent / AVM times 100	times vacancy_indicator rightarrow investment_distres
12	lastSalePrice	price_tier_quintile	Quintile rank within ZIP	s times absorption_rate rightarrow
13	latitude / longitude	haversine_cluster_id	HDBSCAN cluster assignment	times propensity_score rightarrow
14	zipCode	zip_absorption_rate	Joined from market data layer	revenue_density_ times weight dom_trend_slope rightarrow
15	ownerName / mailingAddress	absentee_owner_flag	Mailing neq property address	times distress_raw rightarrow absentee_distress _signal

2.5 Advanced Scoring Model: XGBoost Architecture

The transition from a linear weighted sum to a gradient-boosted tree model is the single highest-impact engineering decision in Phase 2. Research confirms that replacing rule-based systems with XGBoost models significantly enhances lead qualification accuracy, with ML models achieving property valuations up to 95% accuracy and predictive conversion accuracy up to 82.33%.

Table 6 — XGBoost Hyperparameter Recommendations for Propensity Scoring

Hyperparameter	Recommended Value	Rationale
n_estimators	500–1,000	Sufficient trees for complex non-linear interactions; use early stopping
max_depth	4–6	Prevents overfitting on sparse distress signals; captures 3rd-order interactions
learning_rate (eta)	0.01–0.05	Low rate with high n_estimators for stable convergence
subsample	0.7–0.8	Row subsampling reduces variance on imbalanced dataset (1–5% conversion rate)
colsample_bytree	0.6–0.8	Feature subsampling prevents co-adaptation of correlated neighborhood features
scale_pos_weight	15–50	Critical for imbalanced data: ratio of negative to positive class (95:5 to 99:1)
min_child_weight	5–10	Prevents splits on very rare distress signal combinations
reg_alpha (L1)	0.1–1.0	Sparse regularization; drives irrelevant features toward zero weight
reg_lambda (L2)	1.0–5.0	Ridge regularization; stabilizes equity band and LTV feature weights
eval_metric	aucpr	Precision-Recall AUC is primary metric for imbalanced conversion data
tree_method	hist	Histogram-based algorithm; required for 140M+ record processing speed

Model Output and Score Normalization: The XGBoost model outputs a raw probability p in $[0, 1]$ representing the likelihood of a property selling within 90 days. This is normalized to the 0–1,000 scale via: $\text{propensity_score} = \text{round}(p \text{ times } 1000)$. The median score of 500 corresponds to $p = 0.50$. Scores above 700 are classified as High Priority (top decile targeting), 500–700 as Medium Priority, and below 500 as Low Priority. Field reps are routed exclusively to High and Medium Priority properties during Phase 1.

Expected AUC Improvement: Baseline linear weighted sum models typically achieve AUC-ROC of

0.60–0.65 on real estate propensity tasks. XGBoost with the feature set specified above is expected to achieve AUC-ROC of 0.75–0.82, representing a 15–25% relative improvement. The Precision-Recall AUC (primary metric for imbalanced data) is expected to improve from approximately 0.08 (random baseline at 5% conversion rate) to 0.25–0.35, representing a 3–4times lift in actionable lead identification.

3. DYNAMIC WEIGHT CALIBRATION

Static propensity models degrade rapidly in real estate markets due to interest rate volatility, seasonal cycles, and geographic regime shifts. Algorithm II implements a three-layer dynamic calibration system: Bayesian weight updating from field rep feedback, Hidden Markov Model market regime detection, and a continuous MLOps pipeline with automated drift detection and retraining triggers.

3.1 Bayesian Updating Framework

The Bayesian calibration layer treats each weight configuration as a hypothesis and updates its posterior probability as field reps log conversion outcomes. The mathematical foundation is Bayes' Theorem: $P(\theta|data) \propto P(data|\theta) \times P(\theta)$, where θ represents the feature weight vector and data represents observed conversion outcomes.

Beta-Binomial Model Specification: For binary conversion outcomes (homeowner lists or does not list), the Beta distribution is the conjugate prior for the Binomial likelihood. This enables closed-form posterior updates without numerical integration. The prior is initialized as $Beta(\alpha, \beta)$, where α is a number of prior conversions + 1 and β is a number of prior non-conversions + 1. A weakly informative prior of $Beta(3, 17)$ encodes a prior belief of approximately 15% conversion rate. After observing field data with C conversions and N non-conversions, the posterior updates as: $\alpha_{new} = \alpha + C$ and $\beta_{new} = \beta + N$. Example: Prior $Beta(3, 17)$ + field data (720 conversions, 3,280 non-conversions) $\rightarrow$ Posterior $Beta(723, 3,297)$. The posterior mean conversion rate = $723 / (723 + 3,297) = 0.180$, representing an 18% estimated conversion rate for the tested weight configuration.

Thompson Sampling for Weight Configuration Selection: Multiple weight configurations (arms) are tested simultaneously using Thompson Sampling. At each decision point, a conversion probability p_i is sampled from each arm's $Beta(\alpha_i, \beta_i)$ distribution. The arm with the highest sampled p_i is selected for the next rep's route. This naturally balances exploration (testing undersampled configurations) and exploitation (deploying proven configurations) without requiring explicit epsilon-greedy tuning.

Deconfounded Thompson Sampling (DTS) for Market Shocks: When exogenous factors (interest rate hikes, policy changes, seasonal shifts) confound conversion outcomes, standard Thompson Sampling incorrectly attributes performance changes to weight configurations. DTS addresses this by computing inverse propensity weights for each observation: $IPW_i = 1 / P(\text{arm}_i \text{ selected} | \text{context}_i)$. The weighted update becomes: $\alpha_{new} = \alpha + \sum (IPW_i \times \text{conversion}_i)$ and

$\text{beta_new} = \text{beta} + \epsilon (1 - \text{conversion_i})$. This prevents the algorithm from abandoning effective weight configurations during market downturns caused by external factors.

Field Rep Feedback Integration Protocol: The funnel progression events are mapped to conversion signals as follows: knocked = impression (no update), answered = soft positive signal (alpha += 0.1), interested = strong positive signal (alpha += 0.5), appointment_set = conversion event (alpha += 1.0), no_answer = soft negative (beta += 0.2), rejected = negative signal (beta += 0.5). This granular event weighting allows the Bayesian model to update from partial funnel data rather than waiting for full conversion events, which are rare at 1–5% base rates.

3.2 Market Condition Sensitivity

Market Regime Detection via Hidden Markov Models: HMMs infer unobservable market states (hot, neutral, cold) from observable indicators. The model is specified as a 3-state HMM with emission distributions over the observable vector: $O_t = [\text{absorption_rate}_t, \text{median_dom}_t, \text{list_to_sale_ratio}_t, \text{price_momentum_index}_t]$. Each state has a Gaussian emission distribution $N(\mu_k, \Sigma_k)$ for $k \in \{\text{hot}, \text{neutral}, \text{cold}\}$. The Viterbi algorithm decodes the most likely state sequence. Research confirms that Markov-switching models applied at the MSA level can identify bust phases approximately 2 years earlier than monthly price data alone, providing a critical early warning signal for weight recalibration.

Table 7 — Market Regime Detection Thresholds and Conditional Weight Adjustment Rules

Market Regime	Absorption Rate	Median DOM	List-to-Sale Ratio	Weight Adjustment Rule
Hot (Seller's Market)	> 20%	< 21 days	> 1.02	Increase weight on absorption_rate_delta (+20%), dampen distress signals (-15%), boost price_momentum_index (+25%)
Neutral (Balanced)	15–20%	21–45 days	0.97–1.02	Use baseline calibrated weights; no conditional
Cold (Buyer's Market)	< 15%	> 45 days	< 0.97	Increase weight on distress signals (+30%), boost life_event_score (+20%), dampen price_momentum (-20%)

Transitioning (HMM uncertainty)	Mixed signals	High variance	Volatile	Apply ensemble of hot/cold weights weighted by HMM state posterior probabilities
------------------------------------	---------------	---------------	----------	--

Seasonal Adjustment Implementation: House price seasonality has increased significantly over the past decade, requiring MSA-specific seasonal indices for 400+ metropolitan areas. The seasonal adjustment factor (SAF) is applied as a multiplicative modifier to the `spring_selling_season` feature weight: `adjusted_weight = base_weight times SAF_month_MSA`. SAF values are pre-computed from historical transaction data: SAF peaks at 1.25–1.40 in April–June (spring selling season) and troughs at 0.65–0.80 in December–January. Urban MSAs exhibit stronger seasonality than rural markets, requiring distinct baseline SAF tables.

3.3 Model Validation Framework

Holdout Set Design: Temporal holdouts are strictly required for real estate propensity models. Random holdouts introduce data leakage by allowing future market conditions to inform past predictions. The recommended split is: training on months T-24 through T-3, validation on months T-3 through T-1, and test on month T (most recent). This simulates real-world deployment where the model is always predicting future conversions from past data.

Table 8 — Complete Validation Stack for Algorithm II Propensity Model

Metric	Formula / Definition	Target Threshold	Application in FirstKnock
AUC-ROC	Area under ROC curve; $P(\text{score_positive} > \text{score_negative})$	> 0.75	Overall ranking ability; primary model selection metric
Precision-Recall AUC	Area under Precision-Recall curve	> 0.25	Primary metric for imbalanced data (1–5% conversion rate)
KS Statistic	$\max \text{CDF_positive}(t) - \text{CDF_negative}(t) $	> 0.35	Separation between converted and non-converted score distributions
Brier Score	$\text{Mean}((\text{predicted_prob} - \text{actual_outcome})^2)$	< 0.05	Probabilistic calibration accuracy; lower is better
Lift at Top Decile	Conversion rate in top 10% / Overall conversion rate	> 3.0times	Prioritization efficiency for field rep routing

Lift at Top Quintile	Conversion rate in top 20% / Overall conversion rate	> 2.5times	Practical routing threshold for daily work orders
Calibration Plot	Predicted probability vs. actual conversion rate by decile	Max deviation < 5%	Ensures score of 700 truly means ~70th percentile propensity

3.4 Continuous Learning Pipeline

Drift Detection Mechanisms: Two distinct drift types require monitoring. Feature Drift (input distribution shift) is tracked using the Population Stability Index: $PSI = \sum (Actual\% - Expected\%) \times \ln(Actual\% / Expected\%)$. PSI thresholds: $PSI < 0.10$ = no action, $0.10-0.25$ = monitor closely, $PSI > 0.25$ = trigger retraining. Concept Drift (degradation of feature-to-target relationship) is detected using ADWIN (Adaptive Windowing), which maintains a sliding window of conversion outcomes and triggers an alert when the mean conversion rate in the recent window differs significantly from the historical window. The Page-Hinkley test provides a complementary sequential detection method for gradual concept drift.

MLOps Pipeline Architecture: The production pipeline uses Evidently for automated drift monitoring (PSI computation, data quality checks, distribution shift reports) and Metaflow for pipeline orchestration (DAG-based step execution, artifact versioning, compute scaling). MLflow tracks all model versions, hyperparameter configurations, and validation metrics. Retraining is triggered by performance-based conditions, not time-based schedules.

Table 9 — Retraining Trigger Conditions and Priority Levels

Trigger Condition	Threshold	Action	Priority
AUC-ROC drop	< 0.72 on rolling 7-day validation	Immediate full retrain	P0 — Critical
PSI feature drift	> 0.25 on any top-5 feature	Retrain within 24 hours	P1 — High
ADWIN concept drift alert	Mean conversion rate shift > 2sigma	Retrain within 48 hours	P1 — High
Lift at top decile drop	< 2.0times for 3 consecutive days	Investigate + retrain	P2 — Medium
Market regime transition	HMM state change detected	Conditional weight recalibration only	P2 — Medium
Scheduled maintenance	Monthly	Full retrain + hyperparameter sweep	P3 — Low

Geospatial clustering transforms the scored property universe into actionable, revenue-optimized field territories. The goal is not geographic balance — it is revenue density maximization subject to daily capacity constraints. Nearly two-thirds of B2B companies find their territory design ineffective; Algorithm II addresses this through propensity-weighted clustering, contiguous zone enforcement, and rigorous quality validation.

4.1 Algorithm Selection Guide

Table 10 — Clustering Algorithm Selection Guide for FirstKnock Territory Design

Algorithm	Strengths	Weaknesses	Use Case in FirstKnock	Production Recommendation
K-Means	Computationally efficient; easy to implement; deterministic with K-Means++ init	Struggles with varying densities and outliers; requires pre-specifying K	Phase 1 baseline territory sizing; initial segmentation of uniform suburban grids	Phase 1 only
DBSCAN	Handles noise effectively; identifies arbitrary cluster shapes; no K required	Requires strict tuning of eps parameter; struggles with varying densities across geographies	Dense urban grid clustering with relatively uniform property density	Phase 1 urban markets
HDBSCAN	No eps required; handles varying densities; robust to outliers; supports approximate_predi	Computationally heavier than DBSCAN; non-deterministic across runs	Production clustering for all mixed suburban/urban/rural markets with varying property	Phase 2+ PRODUCTION
Propensity-Weighted K-Means	ct for new points Integrates revenue signal into geographic clustering; sklearn sample_weight support	Inherits K-Means density limitations; requires propensity scores as prerequisite	densities Revenue-optimized territory design after propensity scores are available	Phase 2+ PRODUCTION

REDCAP/MDD	Enforces geographic contiguity; optimal edge removal via dynamic programming	Computationally expensive; requires PySAL spatial weights matrix	Final contiguous zone enforcement after density clustering	Phase 2+ PRODUCTION
------------	--	--	--	---------------------

4.2 DBSCAN and HDBSCAN Configuration

Epsilon (eps) Selection via K-Distance Graph: For DBSCAN, compute the k-nearest neighbor distances for all points ($k = \text{MinPts}$), sort in ascending order, and plot. The optimal eps corresponds to the 'elbow' — the point of maximum curvature where distances sharply increase. This threshold separates cluster points from noise. In practice, use kneed library's KneeLocator with `curve='convex'`, `direction='increasing'` to automate elbow detection.

Table 11 — DBSCAN Parameter Recommendations by Geography Type

Geography Type	Recommended eps (km)	Recommended MinPts	Rationale
Dense Urban (e.g., NYC, Chicago core)	0.15–0.25 km	8–12	Small radius prevents mega-clusters; high MinPts filters noise in dense grids
Suburban (e.g., typical metro suburbs)	0.40–0.70 km	5–8	Moderate radius captures neighborhood-level clusters; balanced noise filtering
Sparse Suburban / Exurban	0.80–1.50 km	3–5	Larger radius prevents over-fragmentation; lower MinPts captures sparse clusters
Rural	2.00–5.00 km	3–4	Very large radius required; accept lower cluster density; focus on high-propensity outliers

HDBSCAN Production Implementation: HDBSCAN is the production recommendation for Phase 2+. Key parameters: `min_cluster_size = max(5, int(0.001 times n_properties_in_territory))` to scale with dataset size; `min_samples = 3–5` for noise robustness; `metric = 'haversine'` for accurate great-circle distance calculations on lat/lon coordinates (input must be in radians: `coords_rad = np.radians(coords_deg)`); `cluster_selection_method = 'eom'` (Excess of Mass) for stable cluster

boundaries. HDBSCAN's approximate_predict function enables scoring of new properties added between full re-clustering runs, critical for the daily delta update pipeline.

4.3 Propensity-Weighted Clustering

Standard geographic clustering treats all properties equally. Propensity-weighted clustering biases cluster formation toward high-propensity concentrations, ensuring that territory boundaries maximize revenue density rather than geographic coverage.

Modified Objective Function: Standard K-Means minimizes $\sum_i \|x_i - \mu_b\|^2$.

Propensity-weighted K-Means minimizes $\sum_i w(x_i) \|x_i - \mu_b\|^2$ where $w(x) = \text{propensity_score}(x) / \text{mean_propensity_score}$. This is implemented directly in sklearn via KMeans with sample_weight=propensity_scores parameter. The weighted centroid update becomes: $\mu_b \leftarrow \frac{\sum_i w(x_i) \cdot x_i}{\sum_i w(x_i)}$, pulling cluster centers toward high-propensity property concentrations.

Getis-Ord G_i^* Hotspot Analysis: Before clustering, apply Getis-Ord G_i^* via PySAL to identify statistically significant spatial concentrations of high propensity. The G_i^* statistic for location i is: $G_i^*(d) = \frac{(\sum_j w_{ij} \cdot x_j) - \bar{x} \sum_j w_{ij}}{\sqrt{S^2 \sum_j w_{ij} (1 - w_{ij})}}$ where $w_{ij}(d)$ is the spatial weight matrix, x_j is the propensity score at location j , $\bar{x}$ is the mean propensity score, and S is the standard deviation. Locations with G_i^* z-score > 1.96 ($p < 0.05$) are classified as statistically significant hotspots. These hotspot zones receive priority routing in the daily work order, independent of cluster assignment.

KDE Revenue Density Surface: A Kernel Density Estimation surface is computed over propensity-weighted property locations using a Gaussian kernel with bandwidth selected via Scott's rule: $h = n^{-1/5}$ times sigma. The KDE surface is rasterized to a 100m times 100m grid and used as a visual overlay in the field rep app to communicate revenue density to reps in the field.

4.4 Territory Design Optimization

Capacitated Clustering Constraint: Each territory (cluster) must contain between 40 and 60 high-priority properties per rep per day. This constraint is enforced via the p-median formulation: minimize $\sum_j w_j \cdot d_{ij} \cdot x_{ij}$ subject to: $\sum_i x_{ij} = 1$ for all j (each property assigned to exactly one territory), $\sum_j x_{ij} \leq \text{capacity_max}$ for all i (capacity constraint), $\sum_i y_i = p$ (exactly p territories), where w_j is the propensity weight of property j , d_{ij} is the distance from property j to territory center i , x_{ij} is the assignment binary variable, and y_i indicates if location i is a territory center.

REDCAP Contiguous Zone Enforcement: After capacitated clustering, REDCAP (Regionalization with Dynamically Constrained Agglomerative Clustering and Partitioning) enforces geographic contiguity. The algorithm builds a minimum spanning tree of the spatial weights graph, then optimally removes $p-1$ edges to create p contiguous zones. The MDD (Multi-Valued Decision Diagram) enhancement finds the globally optimal edge removal sequence via dynamic

programming, producing higher-quality partitions than greedy approaches. Implementation: from pysal.region import MaxPHeuristic; use spatial_weights = Queen.from_dataframe(gdf) to construct the contiguity matrix.

Pareto Frontier for Distance-Propensity Trade-off: Territory design involves a fundamental trade-off between minimizing travel distance (rep efficiency) and maximizing propensity concentration (revenue density). The Pareto frontier is computed by solving the p-median problem with varying propensity weight multipliers lambda in [0, 1]: objective = lambda times propensity_score_sum + (1-lambda) times ("total_travel_distance"). The recommended operating point is lambda = 0.65, prioritizing revenue density while maintaining reasonable travel efficiency. This parameter is exposed as a configurable setting in the territory design dashboard.

4.5 Cluster Quality Validation

Table 12 — Cluster Quality Validation Metrics with Acceptance Thresholds

Metric	Formula	Target Threshold	Re-clustering Trigger
Davies-Bouldin Index	$\text{Mean}(\max_j \frac{\text{sig}(\text{max}_i d(i, c_j))}{\text{sig}(\text{max}_i d(i, c_i))})$	< 0.50	DBI > 0.75 triggers full re-cluster
Calinski-Harabasz Score	$\frac{d(c_k, c_l)}{d(c_k, c_l)}$ Between-cluster dispersion / Within-cluster dispersion times	> 150	CH < 100 triggers parameter re-tuning
Silhouette Score	$s(i) = (b(i) - a(i)) / \max(a(i), b(i))$	> 0.40	Mean silhouette < 0.30 triggers re-cluster
Dunn Index	min inter-cluster distance / max intra-cluster diameter	> 0.30	Dunn < 0.20 triggers re-cluster
Intra-cluster Propensity Variance	$\text{Var}(\text{propensity_scores within cluster}) / \text{Global_Var}$	< 0.10	Ratio > 0.20 indicates heterogeneous territory
Capacity Balance Ratio	$\max(\text{cluster_size}) / \min(\text{cluster_size})$	< 1.5	Ratio > 2.0 triggers rebalancing

Production Requirement: All six cluster quality metrics must pass acceptance thresholds before a territory assignment is pushed to the field rep application. Automated validation runs nightly as part of the clustering pipeline. Any metric failure triggers an alert to the ML engineering team and blocks the territory update until re-clustering is complete.

5. TSP/ROUTING OPTIMIZATION

Route optimization is the final transformation layer that converts a ranked list of high-propensity

properties into an executable daily work order. The routing engine must balance travel efficiency, rep fatigue, scheduled breaks, and real-time adaptation to conversion events. The target is a 15–20% reduction in total travel distance versus naive nearest-neighbor routing, with routes structured as loops to guarantee Christofides-bounded optimality.

5.1 Route Construction Algorithm

Construction Phase — Nearest Neighbor Heuristic: The initial route is constructed using the nearest-neighbor greedy heuristic: starting from the rep's home/depot location, iteratively visit the unvisited property with the minimum Haversine distance. This $O(n^2)$ construction produces a feasible but suboptimal tour that serves as the input to 2-Opt refinement.

Refinement Phase — 2-Opt Algorithm: The 2-opt algorithm eliminates route crossings through iterative edge swaps. For a tour with edges $(t1, t2)$ and $(t3, t4)$, a 2-opt move removes these edges and reconnects as $(t1, t3)$ and $(t2, t4)$. The swap executes if and only if: $distance(t1, t3) + distance(t2, t4) < distance(t1, t2) + distance(t3, t4)$. The lengthDelta calculation requires only $O(1)$ operations for symmetric distance matrices: $lengthDelta = d(t1,t3) + d(t2,t4) - d(t1,t2) - d(t3,t4)$. The algorithm iterates through all (i, j) edge pairs until no improving swap exists, reaching a local optimum. For $n = 50$ properties (daily route), the worst-case iterations are $O(n^2) = 2,500$, completing in milliseconds.

Or-Opt for Fast Local Search: Or-Opt is applied after 2-Opt as a secondary refinement. It relocates sequences of 1, 2, or 3 consecutive nodes to better positions in the tour without reversing segments. Or-Opt is faster than 2-Opt per iteration and often finds improvements that 2-Opt misses, particularly for clustered property layouts. The combined 2-Opt + Or-Opt pipeline typically achieves solutions within 3–5% of optimal for $n \leq 60$.

Table 13 — Route Optimization Algorithm Comparison

Algorithm	Complexity	Optimality Guarantee	Use in Pipeline
Nearest Neighbor	$O(n^2)$	None (greedy)	Initial tour construction
2-Opt	$O(n^2)$ iterations, $O(1)$ per swap	Local optimum	Primary refinement layer
Or-Opt	$O(n^2)$ iterations	Local optimum (different neighborhood)	Secondary refinement layer
Christofides	$O(n^3)$	$\leq 1.5 \times$ optimal for cycles	Benchmark and fallback for large territories
Lin-Kernighan	$O(n^{2.2})$ empirical	Near-optimal in practice	Phase 3 upgrade for high-value territories

5.2 Loops Not Lines Implementation

Mathematical Case for Loop Topology: The Christofides algorithm provides the theoretical

foundation for preferring circular routes. For cycle (loop) TSP instances, Christofides guarantees a tour length $< 3/2$ times optimal. For path-based TSP with two specified endpoints (linear routes), the performance ratio degrades to up to $5/3$ times optimal — a 11% relative degradation. For field sales with 40–60 stops, this translates to 2–4 additional unnecessary miles per shift. Loop routes also reduce cognitive load for reps, as they return naturally to their starting point without backtracking.

Loop Enforcement in TSP Formulation: Loop topology is enforced by: (1) setting the depot node as both the start and end node in the OR-Tools routing model, (2) prohibiting the use of the direct depot-to-depot edge (forcing the route to traverse all property nodes), and (3) applying the 'add_dimension' constraint in OR-Tools to enforce that the route forms a Hamiltonian cycle. For large territories ($n > 60$ properties), petal/sector decomposition is applied: the territory is divided into angular sectors from the depot centroid, and a separate loop route is constructed for each sector, with reps assigned to sectors based on daily capacity.

Cognitive Routing Alignment: Research confirms that humans solving TSP in non-Euclidean environments rely on hierarchical clustering and Multidimensional Scaling (MDS) rather than strict Euclidean distances. The Algorithm II routing output is pre-processed through a hierarchical cluster ordering (visit all properties in cluster A before moving to cluster B) to align with natural human navigation intuition, reducing rep cognitive load and improving adherence to the optimized route.

5.3 Break Point Optimization

VRPTW Formulation for Scheduled Breaks: Scheduled breaks are modeled as virtual nodes injected into the property graph. Each virtual break node v_b has: `service_time` = 30 minutes (lunch) or 10 minutes (short break), `time_window` = [`earliest_break_time`, `latest_break_time`], zero travel cost to/from the nearest amenity location (restaurant, coffee shop, park). The Vehicle Routing Problem with Time Windows (VRPTW) formulation minimizes: $\sum_{i,j} c_{ij} x_{ij}$ subject to: `time_window` constraints for all nodes (including virtual break nodes), capacity constraints (40–60 property visits per route), and route duration constraints (maximum 9-hour shift).

VRPTW with Duration Constraints (VRPTWDC): The VRPTWDC variant enforces strict route duration limits via a branch-cut-and-price algorithm. This is mathematically analogous to enforcing driver working hour regulations. Implementation in Google OR-Tools:

`routing.AddDimension(transit_callback_index, max_wait_time=3600, max_route_duration=32400, fix_start_cumul_to_zero=True, name='Time')`. The 32,400-second (9-hour) duration constraint prevents routes from exceeding safe working hours, directly supporting the fatigue modeling layer.

Amenity-Aware Break Placement: Virtual break nodes are placed at the geographically optimal point in the route (minimizing detour distance) subject to the constraint that the break location must be within 0.5 km of a rated amenity (restaurant rating ≥ 3.5 stars, or public park). Amenity data is sourced from the Google Places API and cached weekly. The break placement algorithm selects the route segment $(i, i+1)$ where inserting the virtual break node minimizes: $\text{detour_cost} = d(i, v_b) + d(v_b, i+1) - d(i, i+1)$.

5.4 Real-Time Route Adaptation

Dynamic Re-optimization Triggers: Real-time route updates are triggered by the following field

events: (1) Conversion event — rep sets appointment at property P; nearby high-propensity properties within 0.5 km radius are immediately promoted to the active route. (2) No-answer cluster — three consecutive no-answers in a geographic micro-cluster triggers a skip-and-reroute to the next highest-propensity cluster. (3) Traffic event — travel time to next node exceeds 2times estimated time; incremental 2-Opt re-optimization is triggered on the remaining route. (4) Early completion — rep finishes assigned route with > 45 minutes remaining; nearest unassigned high-propensity cluster is appended.

Incremental 2-Opt for Partial Updates: Full TSP re-optimization on route change is computationally unnecessary. Incremental 2-Opt restricts the edge-swap search to a local neighborhood of the changed node: only edges within a 1.0 km radius of the inserted/removed node are considered for swapping. This reduces the search space from $O(n^2)$ to $O(k^2)$ where k is the number of nodes in the local neighborhood (typically $k = 5-10$), enabling sub-second route updates on mobile devices.

RL-AVNS for Complex Dynamic Scenarios: For Phase 3, Reinforcement Learning Adaptive Variable Neighborhood Search (RL-AVNS) replaces incremental 2-Opt for complex multi-event scenarios. A transformer-based neural policy network selects the optimal neighborhood operator (2-Opt, Or-Opt, VRPTW re-solve) based on the current route state, remaining time, and conversion event history. The fitness metric quantifies temporal flexibility — routes with more slack time receive more aggressive re-optimization. RL-AVNS demonstrates superior performance in dynamic scenarios versus static heuristics.

Route Delta API Specification: Real-time route updates are delivered to the field rep app via a Route Delta API endpoint. POST /api/v1/route/delta with payload: {rep_id, trigger_event, current_position, completed_stops, remaining_stops}. Response: {delta_type: 'insert'|'remove'|'reorder', affected_stops: [...], new_sequence: [...], estimated_time_delta_minutes: N, new_total_distance_km: X}. The API must respond within 2 seconds to maintain rep workflow continuity.

5.5 Rep Fatigue Modeling

Biomathematical Fatigue Constraints: Traditional Hours of Service (HOS) regulations are mathematically insufficient for capturing actual fatigue risk. Algorithm II integrates Biomathematical Fatigue Constraints (BFCs) into the routing layer. The energy depletion model tracks a rep's fatigue state $E(t)$ over the shift: $dE/dt = \alpha \cdot walking_speed_temp_factor(t) + \gamma \cdot consecutive_rejections(t) + \delta \cdot break_recovery(t)$. Parameters: $\alpha = 0.02$ (base depletion rate per minute walking), $\beta = 0.005 \times \max(0, temperature_celsius - 25)$ (heat stress factor), $\gamma = 0.01$ per consecutive rejection (psychological fatigue), $\delta = 0.15$ per minute of break (recovery rate).

Performance Degradation Function: Conversion probability degrades as a function of fatigue state: $conversion_rate_adjusted(t) = conversion_rate_base \times (1 - \max(0, (E_max - E(t)) / E_max)^{1.5})$. This cubic degradation function reflects research showing that performance decline accelerates non-linearly as fatigue accumulates. When $E(t)$ drops below 30% of E_max , the routing engine automatically triggers a mandatory break insertion and reduces the remaining route by 20% to prevent severe performance degradation.

Table 14 — Optimal Shift Structure with Fatigue-Adjusted Conversion Rate Expectations

Shift Period	Recommended Structure	Expected Conversion Rate	Fatigue Level
Hours 1–2 (Morning)	High-propensity cluster, dense routing, 12–15 stops/hour	Baseline times 1.15	Low (E > 80%)
Hours 3–4 (Mid-Morning)	Moderate-propensity cluster, standard routing, 10–12 stops/hour	Baseline times 1.05	Moderate (E 60–80%)
Hour 4.5 (Lunch Break)	30-minute break at rated amenity,	N/A	Recovery to E > 70%
Hours 5–6 (Early Afternoon)	mandatory High-propensity cluster (second best), 10–12 stops/hour	Baseline times 1.00	Moderate (E 55–70%)
Hours 7–8 (Late Afternoon)	Medium-propensity cluster, reduced pace, 8–10 stops/hour	Baseline times 0.90	High (E 35–55%)
Hour 8.5 (Short Break)	10-minute break, mandatory if E < 40%	N/A	Recovery to E > 45%
Hour 9 (End of Shift)	Nearest-to-depot high-propensity stops only, 6–8 stops	Baseline times 0.80	High (E 25–40%)

Circadian Rhythm Adjustment: Conversion rates exhibit circadian patterns independent of fatigue. Research on human performance rhythms indicates peak cognitive performance at 10:00–12:00 and 15:00–17:00, with a post-lunch trough at 13:00–14:30. The routing engine applies a `circadian_adjustment_factor` to the expected conversion rate: peak hours (10–12, 15–17) receive a +10% boost, trough hours (13–14:30) receive a " 1 5 % penalty. High-propensity properties are preferentially scheduled during peak performance windows. Workload balancing via multi-objective genetic algorithms with fuzzy clustering preprocessing has demonstrated travel time reductions of up to 69% compared to baseline methods, directly reducing physical exertion.

6. INTEGRATED IMPLEMENTATION ROADMAP

Phase 1: Foundation (Weeks 1–6)

Phase 1 establishes the core data pipeline and delivers a functional (if not yet optimally intelligent) version of Algorithm II to field reps. The goal is a working end-to-end system that generates scored, clustered, and routed daily work orders within 6 weeks.

Table 15 — Phase 1 Deliverables and Success Criteria

Week	Deliverable	Owner	Success Criteria
1–2	RentCast API integration and feature extraction pipeline	Data Engineering	140M+ records ingested; top-15 fields extracted; PostGIS indexed
2–3	Baseline feature engineering (decay functions, absorption rate, LTV, life events)	ML Engineering	All 15 features computed; RobustScaler normalization applied;
3–4	Enhanced weighted sum propensity scoring (pre-XGBoost baseline)	ML Engineering	Scores generated for all properties; 0–1,000 normalization; AUC > 0.62 on holdout
4–5	DBSCAN/K-Means clustering implementation with Haversine metric	ML Engineering	Territories generated; Davies-Bouldin < 0.75; capacity constraint 40–60 houses enforced
5–6	2-Opt route optimization with OR-Tools integration	Backend Engineering	Routes generated in < 15 min per rep; loop topology enforced; 10%+ distance reduction vs. naive

Phase 2: Intelligence Layer (Weeks 7–14)

Phase 2 replaces the baseline scoring and clustering components with production-grade ML models and introduces real-time adaptation capabilities. This phase delivers the full Algorithm II intelligence stack.

Table 16 — Phase 2 Deliverables and Success Criteria

Week	Deliverable	Owner	Success Criteria
7–8	XGBoost/LightGBM propensity model training and validation	ML Engineering	AUC-ROC > 0.75; Lift at top decile > 3.0times; Brier Score < 0.05
8–9	Bayesian Beta-Binomial weight calibration with Thompson Sampling	ML Engineering	Real-time Beta parameter updates < 5s; DTS confounding correction validated

9–10	HDBSCAN propensity-weighted clustering with Getis-Ord Gi* hotspot	ML Engineering	Davies-Bouldin < 0.50; Intra-cluster propensity variance < 0.10
10–11	analysis REDCAP/MDD contiguous zone enforcement	ML Engineering	All territories geographically contiguous; capacity balance ratio < 1.5
11–12	VRPTW virtual break node injection with amenity-aware placement	Backend Engineering	Break nodes injected in all routes; amenity within 0.5 km; VRPTWDC duration constraint enforced
12–13	Real-time route adaptation API (incremental 2-Opt, conversion-triggered rerouting)	Backend Engineering	Route delta API response < 2s; conversion-triggered promotion of nearby properties
13–14	Field rep app integration and end-to-end pipeline testing	Full Stack / QA	Daily work order delivered by 06:00; all pipeline stages complete within SLA

Phase 3: Continuous Learning (Weeks 15–24)

Phase 3 transforms Algorithm II from a static deployed model into a continuously learning system that adapts to market conditions, rep behavior patterns, and seasonal cycles without manual intervention.

Table 17 — Phase 3 Deliverables and Success Criteria

Week	Deliverable	Owner	Success Criteria
15–17	Full MLOps pipeline: Evidently drift monitoring, Metaflow orchestration, MLflow tracking	MLOps Engineering	PSI computed daily; ADWIN alerts operational; retraining triggers automated

17–19	HMM market regime detection with conditional weight adjustment rules	ML Engineering	3-state HMM trained on 24 months MSA data; regime transitions trigger weight recalibration within 1 hour
19–21	Biomathematical Fatigue Constraint integration into routing layer	Backend Engineering	E(t) tracked per rep per shift; mandatory break triggers operational; circadian adjustment applied
21–22	A/B testing framework for weight configurations (Thompson Sampling)	ML Engineering	Multiple weight configs tested simultaneously; statistical significance threshold $p < 0.05$
22–23	RL-AVNS real-time route adaptation (Phase 3 upgrade from incremental 2-Opt)	ML Engineering	RL-AVNS outperforms incremental 2-Opt by $> 5\%$ on dynamic scenario benchmarks
23–24	Full system integration testing, performance benchmarking, and production hardening	Full Stack / QA	All Phase 3 success metrics achieved; system stable under 500+ concurrent rep load

7. PERFORMANCE BENCHMARKS & SUCCESS METRICS

> 0.75 vs. 0.62 baseline Target AUC-ROC (Propensity Model)	> 3.0 timesvs. 1.0 times random Target Lift at Top Decile	< 0.50 Clustering quality Target Davies-Bouldin Index	< 0.10 Territory homogeneity Target Intra-cluster Propensity Variance
15–20% Routing efficiency Route Distance Reduction vs. Nearest-Neighbor	25–40% Business impact Conversion Rate Lift vs. Unscored Knocking	20–30% Rep productivity Qualified Conversations per Shift Increase	Up to 69% Research benchmark Travel Time Reduction via Fatigue-Aware Clustering

Table 18 — Algorithm II Performance Benchmarks Across All Three Implementation Phases

Domain	Metric	Phase 1 Target	Phase 2 Target	Phase 3 Target	Research Benchmark
Propensity Model	AUC-ROC	> 0.62	> 0.75	> 0.80	82.33% accuracy
Propensity Model	Precision-Recall AUC	> 0.10	> 0.25	> 0.32	3–4times lift over random
Propensity Model	Lift at Top Decile	> 2.0times	> 3.0times	> 3.5times	Industry standard > 3times
Propensity Model	Brier Score	< 0.08	< 0.05	< 0.04	Lower is better
Clustering	Davies-Bouldin Index	< 0.75	< 0.50	< 0.40	Lower is better
Clustering	Calinski-Harabasz Score	> 80	> 150	> 200	Higher is better
Clustering	Intra-cluster Propensity Variance	< 0.20	< 0.10	< 0.08	Lower is better
Routing	Distance Reduction vs. Nearest-Neighbor	> 8%	> 15%	> 20%	Christofides < 1.5times optimal
Routing	Route Generation Latency	< 15 min/rep	< 10 min/rep	< 5 min/rep	SLA requirement
Routing	Real-time Delta API	N/A	< 2 seconds	< 1 second	UX requirement
Business	Conversion Rate vs. Unscored	> 15%	> 25%	> 35%	1–5% base rate
Business	Qualified Conversations per Shift	> 10% increase	> 20% increase	> 30% increase	Rep productivity KPI
MLOps	Model Retraining Latency	Manual	< 6 hours	< 3 hours	Drift response SLA
MLOps	PSI Monitoring Cadence	Weekly	Daily	Real-time	Drift detection SLA

Business Impact Summary: At a fleet of 100 field reps each conducting 50 daily property visits, a

25% conversion rate improvement translates to approximately 1,250 additional qualified conversations per day. At a 10% appointment-set rate from qualified conversations, this represents 125 additional appointments daily — a compounding revenue impact that scales linearly with rep headcount.

8. TECHNICAL APPENDIX

8.1 Mathematical Reference — Key Formulas Consolidated

Table 19 — Complete Mathematical Formula Reference for Algorithm II

Formula Name	Mathematical Expression	Section Reference
Exponential Ownership Decay	$f(t) = e^{-(\lambda \cdot t)}$ where λ is in {0.005, 0.008, 0.012} by segment	§2.1
Cox Proportional Hazards	$h(t X) = h_0(t) \exp(\beta \cdot \text{location} + \beta \cdot \text{floor} + \beta \cdot \text{aff} + \text{LTV})$	§2.1
Loss Aversion Penalty	$\text{loss_aversion_penalty} = \max(0, (\text{purchase_price} - \text{AVM}) / \text{purchase_price})$	§2.1
Absorption Rate	$\text{absorption_rate} = (\text{Units Sold} / \text{Total Active Inventory}) \times 100$	§2.2
Price Momentum Index	$\text{PMI} = ((\text{median_sale_30d} / \text{median_sale_90d}) - 1) \times 100$	§2.2
Life Event Score	$\text{life_event_score} = \exp(-0.05 \times \text{days_since_event_i})$	§2.3
Equity Band Score	$\text{equity_band_score} = 1 - \min(1, \max(0, (\text{LTV} - 0.20) / 0.80))$	§2.3
Composite Motivation Index	$\text{CMI} = 0.30 \cdot \text{equity_band} + 0.25 \cdot \text{distress_norm} + 0.25 \cdot \text{life_event} + 0.20 \cdot \text{ownership_decay}$	§2.3
Bayes' Theorem (Propensity)	$P(\theta \text{data}) \propto P(\text{data} \theta)$	§3.1
Beta-Binomial Update (alpha)	$\alpha_{\text{new}} = \alpha + \text{conversions}$	§3.1
Beta-Binomial Update (beta)	$\beta_{\text{new}} = \beta +$	§3.1
Inverse Propensity Weight (DTS)	$\text{IPW}_i = 1 / P(\text{arm}_i \text{ selected} \text{context}_i)$	§3.1
Silhouette Score	$s(i) = (b(i) - a(i)) / \max(a(i), b(i))$	§4.5

Getis-Ord Gi* Statistic	$Gi^*(d) = \frac{[\sum_{j \in N(d)} w_{ij} \cdot X_j - \sum_{j \in N(d)} w_{ij} \cdot \bar{X}]^2}{[S \cdot (\sum_{j \in N(d)} w_{ij}^2 - (\sum_{j \in N(d)} w_{ij})^2 / (n-1))]}$	§4.3
Propensity-Weighted K-Means Objective	minimize $\sum_{i \in O(x)} \frac{1}{ x } \sum_{j \in O(x)} x_j - \mu_b ^2$	§4.3
2-Opt Improvement Condition	$d(t1,t3) + d(t2,t4) < d(t1,t2) + d(t3,t4)$	§5.1
2-Opt Length Delta (O(1))	$lengthDelta = d(t1,t3) + d(t2,t4) - d(t1,t2) - d(t3,t4)$	§5.1
Christofides Guarantee (Cycles)	$tour_length < (3/2) \text{ times optimal}$	§5.2
Fatigue Energy Depletion	$dE/dt = \alpha \cdot walk_speed \cdot beta \cdot temp_factor + gamma \cdot rejections +$	§5.5
Performance Degradation	$delta \cdot break_recovery_conversion_adjusted(t) = conversion_base \text{ times } (1 - \max(0, (E_max - E(t)) / E_max))$	§5.5
Population Stability Index	$PSI = \sum_{i=1}^n Actual_i - Expected_i \text{ times } \ln(Actual_i / Expected_i)$	§3.4
KDE Bandwidth (Scott's Rule)	$h = n^{-1/5} \text{ times } \sigma$	§4.3

8.2 Python Library Reference with Version Recommendations

Table 20 — Python Library Reference with Version Requirements and Use Cases

Library	Version	Install Command	Primary Use in Algorithm II
xgboost	geq 2.0.0	pip install xgboost>=2.0.0	Propensity scoring model (tree_method='hist' for scale)
lightgbm	geq 4.0.0	pip install lightgbm>=4.0.0	Alternative/ensemble propensity model
scikit-learn	geq 1.4.0	pip install scikit-learn>=1.4.0	HDBSCAN, K-Means, preprocessing, validation metrics

hdbscan	geq 0.8.33	pip install hdbscan>=0.8.33	Standalone HDBSCAN with approximate_predict support
pysal	geq 23.1	pip install pysal>=23.1	Getis-Ord Gi*, REDCAP, spatial
geopandas	geq 0.14.0	pip install geopandas>=0.14.0	weights matrix Geospatial data manipulation, PostGIS integration
ortools	geq 9.8.0	pip install ortools>=9.8.0	VRPTW solver, 2-Opt, virtual break node injection
scipy	geq 1.12.0	pip install scipy>=1.12.0	Beta-Binomial distributions, Thompson Sampling, KDE
numpy	geq 1.26.0	pip install numpy>=1.26.0	Haversine calculations, matrix operations, decay functions
pandas	geq 2.0.0	pip install pandas>=2.0.0	Feature engineering, rolling windows, data manipulation
hmmlearn	geq 0.3.0	pip install hmmlearn>=0.3.0	Hidden Markov Model market regime detection
evidently	geq 0.4.0	pip install evidently>=0.4.0	PSI drift monitoring, data quality reports
metaflow	geq 2.10.0	pip install metaflow>=2.10.0	MLOps pipeline orchestration, DAG execution
mlflow	geq 2.10.0	pip install mlflow>=2.10.0	Model versioning, experiment tracking, A/B configs
kneed	geq 0.8.5	pip install kneed>=0.8.5	Automated elbow detection for DBSCAN eps selection

8.3 RentCast API Field Mapping to Feature Names

Table 21 — RentCast API Field Mapping to Algorithm II Feature Names with Null Handling

RentCast API Field	API Endpoint	Derived Feature Name	Data Type	Null Handling Strategy
lastSaleDate	/properties/{id}	ownership_duration_months	Float	Impute with ZIP median ownership duration
lastSalePrice	/properties/{id}	purchase_price	Float	Impute with tax assessed value times 1.05
estimatedValue	/properties/{id}	current_avm	Float	Required field; reject record if null
loanAmount	/properties/{id}	outstanding_mortgage	Float	Impute 0 if property age > 30 years
propertyType	/properties/{id}	property_type_encoded	One-hot	Default to 'SFR' if null
squareFootage	/properties/{id}	size_sqft	Float	Impute with bedrooms times 400 heuristic
yearBuilt	/properties/{id}	property_age_years	Integer	Impute with ZIP median year built
bedrooms	/properties/{id}	bedrooms	Integer	Impute with property_type modal value
bathrooms	/properties/{id}	bathrooms	Float	Impute with bedrooms times 0.75 heuristic
ownerOccupied	/properties/{id}	owner_occupied_flag	Binary	Default to 1 (owner-occupied) if null
latitude	/properties/{id}	lat_rad	Float (radians)	Required field; reject record if null
longitude	/properties/{id}	lon_rad	Float (radians)	Required field; reject record if null
zipCode	/properties/{id}	zip_code	String	Required for neighborhood feature join

rentEstimate	/properties/{id}/rent	monthly_rent_estimate	Float	Impute with ZIP median rent by bedroom count
taxAssessedValue	/properties/{id}	tax_assessed_value	Float	Impute with estimatedValue times 0.85

8.4 Key Risks and Mitigations

Table 22 — Algorithm II Key Risks and Mitigation Strategies

Risk	Probability	Impact	Mitigation Strategy
Concept/Feature Drift from market volatility (interest rate shocks, policy changes)	High	High	Evidently MLOps pipeline tracking PSI daily; ADWIN concept drift detection; performance-based retraining triggers (AUC < 0.72)
Computational bottleneck on 140M+ property records	Medium	High	Mini-batch K-Means for clustering; XGBoost hist tree_method; incremental 2-Opt O(1) evaluations; PostGIS spatial indexing
Highly imbalanced training data (1–5% conversion rate)	High	High	scale_pos_weight in XGBoost; Precision-Recall AUC as primary metric; SMOTE oversampling for minority class in
Geographic contiguity violations in territory design	Medium	Medium	training REDCAP/MDD contiguous zone enforcement; Queen contiguity spatial weights matrix validation post-clustering

Rep route non-adherence reducing feedback signal quality	Medium	Medium	Gamification of route adherence in app; GPS-based actual vs. planned route comparison; adherence score in rep performance dashboard
RentCast API rate limiting or data gaps	Low	High	Local PostGIS cache with 7-day TTL; graceful degradation to cached data; null imputation strategies
Bayesian MAB exploration-exploitation imbalance	Low	Medium	DTS confounding correction; minimum exploration budget of 10% of daily routes; cold-start strategy for new geographies

8.5 Academic and Industry Source Summary

Table 23 — Academic and Industry Source Summary by Domain

Domain	Key Methods	Primary Sources / Frameworks
Ownership Duration Modeling	Cox Proportional Hazards, Exponential Decay, Disposition Effect	Survival analysis literature; behavioral economics (loss aversion)
Neighborhood Feature Engineering	Absorption Rate, DOM Trends, Hedonic Price Indices	PropTech market analytics; MLS transaction data research
Composite Motivation Scoring	Distress Signal Stacking, Wholesale Score, Life Event Proxies	Leadflow AI methodology; 800+ signal propensity systems
ML Propensity Scoring	XGBoost, LightGBM, SVM with Recursive Feature Selection	Mortgage broker lead scoring case study; 95% AVM accuracy research
Bayesian Calibration	Beta-Binomial Conjugate Priors, Thompson Sampling, DTS	Multi-Armed Bandit literature; Bayesian inference textbooks
Market Regime Detection	Hidden Markov Models, Markov-Switching Dynamic Factor Models	MSA-level housing cycle research; 2-year early bust detection studies

Model Validation	AUC-ROC, KS Statistic, Brier Score, Precision-Recall, Lift Curves	Imbalanced classification literature; Realtor.com MLOps practices
Drift Detection	Population Stability Index, ADWIN, Page-Hinkley Test	Evidently open-source; Metaflow MLOps; industry drift monitoring standards
Geospatial Clustering	DBSCAN, HDBSCAN, K-Means++, Mini-batch K-Means	Scikit-learn documentation; PropTech territory design research
Hotspot Analysis	Getis-Ord Gi*, Spatial Autocorrelation (ESDA)	PySAL library; spatial statistics literature
Territory Design	p-median, Capacitated Clustering, REDCAP, MDD	B2B territory design research; 2/3 ineffective territory finding
TSP Routing	2-Opt, Or-Opt, Christofides Algorithm, Lin-Kernighan	Combinatorial optimization literature; OR-Tools documentation
VRPTW / Break Modeling	VRPTW, VRPTWDC, Branch-Cut-and-Price	Vehicle routing literature; Google OR-Tools VRPTW examples
Dynamic Routing	LENS, RL-AVNS, Multi-Armed Bandit Time Windows	Reinforcement learning for VRP; transformer-based neural routing
Fatigue Modeling	Biomathematical Fatigue Constraints, Tabu Search, Genetic Algorithms	Transportation fatigue research; 69% travel time reduction studies

DOCUMENT CONTROL

Document Version	1.0 — Initial Release
-------------------------	-----------------------

Classification	Internal — Engineering & Product Team Only
-----------------------	--

Platform	FirstKnock Sales OS firstknock.online
-----------------	---

Algorithm	Algorithm II: Predictive Propensity
Data Source	RentCast API — 140M+ Property Records
Next Review Date	June 2026 (Post-Phase 2 Completion)
Sections	8 Sections 23 Tables 22 Formulas 4 Domains

This document should be reviewed and updated following Phase 1 completion (Week 6), Phase 2 completion (Week 14), and any major market regime shift detected by the HMM monitoring layer. All formula parameters (lambda decay rates, CMI weights, XGBoost hyperparameters) are initial recommendations subject to empirical calibration against FirstKnock field data.